

Module 2: Lesson 4

Convolutional Neural Networks: Training a CNN



Outline

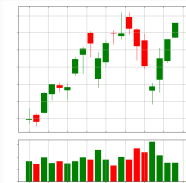
- ▶ Stock return prediction with candlestick charts
- ▶ Network architecture
- ▶ Training the CNN
- ▶ Performance of the model

Stock return prediction with CNNs and candlestick charts

In this last lesson of Module 2, we build and train a fully-fledged, although basic, Convolutional Neural Network.

The objective is to feed the network with images of candlestick patterns, train the architecture, and evaluate its prediction power in the training test.

We exploit a training sample of 1,280 images and a test sample of 360 images that capture candlestick charts of SPY and QQQ.

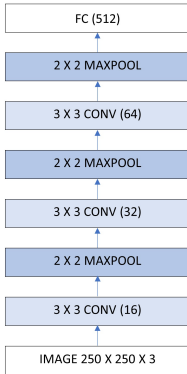


————— "Up" move
— [CNN] — "Up" or "Down" ?



————— "Down" move
— [CNN] — "Up" or "Down" ?

Architecture



The Network architecture shares similarities with AlexNet, by stacking several layers of convolutions and max-pooling.

We increase the width of the layers, i.e., we add channels, as we increase the depth of the network.

After three layers of convolutions + max-pooling, we aggregate all the inputs into a fully connected layer with 512 hidden representations, from where we obtain the last prediction.

Feel free to explore other CNN architectures and check the different prediction abilities of alternative models!

Architecture (some code)

Building a CNN with TensorFlow-Keras turns out to be quite simple.

We keep on adding layers to our network until reaching a flattening layer to introduce (may several) fully-connected layers as in an MLP that allow compiling all the information gathered through the layers to make a prediction.

```
model = tf.keras.Sequential()
# Input layer
model.add(tf.keras.layers.InputLayer(input_shape=(250, 250, 3)))
# Rescaling
model.add(tf.keras.layers.experimental.preprocessing.Rescaling(1./255))
# 1st Convolutional + Max-Pooling layer
model.add(tf.keras.layers.Conv2D(16, 3, activation='relu'))
model.add(tf.keras.layers.MaxPooling2D(2))
# Add more layers in the middle
# nth Convolutional + Max-Pooling layer
model.add(tf.keras.layers.Conv2D(64, 3, activation='relu'))
model.add(tf.keras.layers.MaxPooling2D(2))
# Fully-connected layer
model.add(tf.keras.layers.Flatten())
model.add(tf.keras.layers.Dense(512, activation='relu'))
# Output layer
model.add(tf.keras.layers.Dense(1, activation='sigmoid'))
```

Architecture (adding batch normalization)

We can add batch normalization by inserting layers between the convolution and the activation.

In TensorFlow-Keras, Batch normalization includes two non-trainable parameters, called "momentum" parameters, that smooth the estimation of the current minibatch's mean and variance, using the previous estimate.

Besides, we add a small value to the denominator in batch normalization, 0.001 by default, to avoid division by zero.

```
model.add(tf.keras.layers.Conv2D(16, 5))  
model.add(tf.keras.layers.BatchNormalization())  
model.add(tf.keras.layers.Activation('relu'))
```

Training and performance

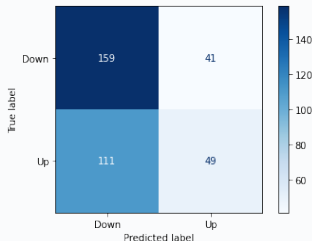
We train the parameters using a standard objective function in binary classification and the RMSprop algorithm.

- We overweight the category with lower frequency ("down") to compensate the lack of balance.

We do not include hyperparameter tuning, so validation and test sets are the same.

- Training is made over 20 epochs and we keep the best model according to its accuracy in validation.
- We see from the optimization how the network begins to overfit without much improvement in validation.

The trained model has some slight predictive ability, although it predicts that prices go "down" most of the time, while in many cases the price goes "up."



Summary of Lesson 4

In Lesson 4, we have:

- ▶ Built a CNN to predict stock prices based on image information from candlestick charts.
- ▶ Trained and evaluated the model's prediction performance.

⇒ **References for this Lesson:**

Chapters 7 & 8 in [Zhang, Aston, et al. "Dive into deep learning." \(2022\).](#)

TO DO NEXT: Please, go to the Jupyter Notebook associated with Lesson 4.

With this, we finalize our introduction to Convolutional Neural Networks. Module 3 will describe further methods meant to improve the convergence and prediction ability of CNNs.